

RESULTS

Exam Results

0 / 1000

NOT PASSED

0 of 60 correct (0%) · Pass mark: 720

Domain Breakdown

|    |                                        |                 |
|----|----------------------------------------|-----------------|
| D1 | Agentic Architecture & Orchestration   | 0/14 (0%) +0pts |
| D2 | Tool Design & MCP Integration          | 0/11 (0%) +0pts |
| D3 | Claude Code Configuration & Workflows  | 0/12 (0%) +0pts |
| D4 | Prompt Engineering & Structured Output | 0/12 (0%) +0pts |
| D5 | Context Management & Reliability       | 0/11 (0%) +0pts |

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes  $0.80 \times 0.27 \times 1000 = 216$  points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

The real exam reports a scaled score from 100 to 1,000 plus your percent-correct by domain, so treat the percentages here as a readiness gauge rather than a prediction of your scaled score.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 60

CORRECT 0

INCORRECT 0

SKIPPED 60

Q01 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-003 ·

Content Moderation and Classification System

SKIPPED

The moderation system uses `tool_use` with a JSON schema for classification output. All fields including `'sub_category'` and `'target_demographic'` are marked as required. Auditors discover that when a post is spam (which has no target demographic), the model fabricates plausible-sounding demographics like `'general public'` or `'young adults.'` What schema change prevents this fabrication?

- A Remove `'target_demographic'` from the schema entirely, on the grounds that its presence is what causes the fabrication in the first place.

Removing the field loses valuable data for categories where target demographic is genuinely relevant (hate speech, harassment). The fix is making it nullable, not removing it.

- B Add a validation step that rejects any `target_demographic` value returned for a post that has been classified as spam, before the record is written.

Post-hoc validation catches the problem but does not prevent it. The model still fabricates values; they are just rejected after the fact. Schema design should prevent fabrication at the source.

- C Add a prompt instruction telling the model to leave `'target_demographic'` empty whenever a post is spam and the field does not apply.

Prompt instructions cannot override a schema that marks the field as required. The JSON schema enforces the constraint at the API level, and the model must comply with the schema regardless of prompt instructions.

- D Make `'target_demographic'` nullable so the model can return null when the field does not apply instead of fabricating a value.

Required fields pressure the model to produce a value even when none exists. Making the field nullable gives the model a legitimate way to indicate 'not applicable,' eliminating the incentive to fabricate. This is the standard pattern for fields that apply conditionally. (correct answer)

WHERE THIS COMES FROM

Lesson 4.3: Structured Output with Tool Use (Nullable fields)

Q02 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-005 · Multi-Agent Research System

SKIPPED

A synthesis agent receives findings from three subagents: solar investment grew 15% (Bloomberg,

January 2024), solar investment grew 22% (IEA, March 2024), and cost-per-watt data. The synthesis agent currently picks the IEA figure because it is more recent. What is the correct approach?

- A Flag the conflicting figures and pause report generation until a human researcher resolves the discrepancy.

Pausing for human resolution is unnecessary when the difference can be explained by temporal or methodological factors. Proper annotation with provenance resolves the ambiguity without blocking the workflow.

- B Average the two values (18.5%) and cite both sources to present a balanced view.

Averaging conflicting statistics is mathematically misleading. The two figures may measure different scopes (e.g., different regions or investment types), making an average meaningless. The difference needs to be preserved, not merged.

- C Present both figures with source attribution and dates, noting what may explain the difference.

Preserving both data points with provenance — source, date, and potential explanations for the difference — lets the consumer interpret the data correctly. Different publication dates and methodologies explain the discrepancy without requiring the synthesis agent to judge which is 'right'. (correct answer)

- D Always use the most authoritative source (IEA as an international agency) and discard the Bloomberg figure.

Discarding a credible source based on perceived authority silently removes valid data. Both sources are credible, and the difference may reflect different methodologies, scopes, or time periods.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q03 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-011 ·

Content Moderation and Classification System

SKIPPED

The coordinator receives a batch of 200 flagged posts to moderate. Currently it processes them sequentially, taking 45 minutes. Each post's moderation is independent — the decision on one post does not affect others. How should the coordinator handle this batch?

- A Increase the iteration cap on the agentic loop to allow the agent more time to process all 200 posts

The iteration cap controls how many tool calls the agent can make within a single task, not how many independent tasks it can process. The issue is parallelism, not loop duration.

- B Process all 200 posts in a single API call by concatenating them into one large prompt

Concatenating 200 posts into a single prompt causes attention dilution. Quality degrades for posts later in the sequence, and a single failure blocks the entire batch.

- C Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete

Since each post's moderation is independent, the coordinator should delegate them to parallel instances. This reduces total processing time from sequential (45 minutes) to roughly the time of the slowest individual moderation, and a failure on one post does not block others. (correct answer)

- D Split into fixed batches of 20 posts and process each batch sequentially in a single prompt

Batching 20 posts into a single prompt still risks attention dilution across posts within each batch. And sequential batch processing still takes much longer than parallel individual processing.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Q04 · D5 5.3 ERROR-PROPAGATION / SILENT-SUPPRESSION q-5-3-006 ·

Enterprise Data Platform with Federated Queries

SKIPPED

The data platform agent queries three sources in sequence: Snowflake (succeeds), PostgreSQL (returns a connection timeout), and a pricing API (succeeds). The agent silently skips the PostgreSQL failure and produces a report using only Snowflake and API data, without informing the user that customer data is missing. What is the critical design failure?

- A The orchestrator should pre-check all data source connections before allowing the agent to begin querying.

Pre-checking connections does not prevent runtime failures (a source can become unavailable between the

check and the query). The agent must handle failures gracefully at query time with structured error propagation.

- B The agent should abort the entire report if any single data source fails, to avoid producing incomplete results.

Aborting on any failure is overly conservative. Partial results with clear coverage annotations are often more useful than no results. The fix is transparency about what is missing, not refusing to produce anything.

- C The agent silently suppresses the error instead of propagating structured error context that annotates which data sources failed and what coverage the report actually has.

Silent error suppression is one of the most dangerous anti-patterns in multi-source systems. The agent must propagate structured error context — identifying which sources failed, what data is missing, and what coverage the final report represents — so the user can make informed decisions about the report's completeness.

(correct answer)

- D The agent should retry the PostgreSQL query automatically with exponential backoff before producing the report.

Automatic retries are appropriate for transient errors, but the critical failure is that the agent silently suppresses the error. Even after retries exhaust, the agent must surface the failure to the user rather than producing an incomplete report without disclosure.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Silent suppression anti-pattern)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q05 · D4 4.1 SYSTEM-PROMPTS / EXPLICIT-CRITERIA q-4-1-006 · Content Moderation and Classification System

SKIPPED

The moderation system's prompt instructs Claude to 'be conservative when moderating and err on the side of caution.' Reviewers find that innocuous posts about cooking with knives, news articles about violence, and fictional war stories are all being flagged as policy violations. What is the root cause and fix?

- A Add an allowlist of safe topics such as cooking, news and fiction that should never be flagged at all by the classifier.

Allowlists are brittle and impossible to maintain comprehensively. New safe topics will constantly appear. The fix

is better criteria that teach the model to distinguish context, not a growing list of exceptions.

- B** The model is too sensitive, so lower the temperature until it stops over-flagging borderline posts in the moderation queue.

Temperature affects randomness, not the interpretation of moderation criteria. The over-flagging is caused by vague instructions, not sampling behaviour.

- C** Add a second moderation pass that re-reads each flagged post under the same 'be conservative' guidance and strips out the false positives before any action.

A second pass using the same vague 'be conservative' criteria will reproduce the same false positives. Fix the criteria first; add verification layers second.

- D** Replace 'be conservative' with explicit categorical criteria defining each violation category with concrete examples.

Vague directives like 'be conservative' give the model no basis for distinguishing between genuine violations and legitimate content that mentions sensitive topics. Explicit criteria with examples of both violations and non-violations (news, fiction, educational content) enable consistent, calibrated decisions. (correct answer)

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Explicit criteria over hedge phrases)

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q06 · D4 4.4 VALIDATION-RETRY / DETECTED-PATTERNS q-4-4-004 ·

Content Moderation and Classification System

SKIPPED

A moderation classification tool includes a `detected\_patterns` array where the model lists specific patterns it identified (e.g. 'repeated slur targeting ethnicity'). Validation checks whether the detected patterns match the assigned category, and on mismatch retries with feedback like 'You classified this as spam but detected patterns of hate speech targeting ethnicity, please re-evaluate.' What is the primary benefit of the `detected\_patterns` field in this workflow?

- A** It increases classification accuracy by forcing the model to identify specific patterns before assigning a category

While structured reasoning fields can improve accuracy, this framing misses the retry workflow context. The primary benefit is enabling programmatic validation of reasoning consistency and targeted feedback for self-correction.

- B** It lets validation detect reasoning inconsistencies and feed back targeted errors the model uses to self-correct.

The detected\_patterns field externalises the model's reasoning into structured data that can be programmatically validated against the conclusion. When patterns and category are inconsistent, the specific error feedback ('you detected hate speech patterns but classified as spam') gives the model actionable information to self-correct. This is the core benefit in a retry-with-error-feedback workflow. (correct answer)

- C** It lets the system auto-correct the category by overriding the model's classification with a rule-based pattern match

The system does not override the model's classification. It feeds back the inconsistency and asks the model to re-evaluate. The model may correct its category or its patterns — the point is self-correction, not rule-based override.

- D** It provides an auditable evidence trail showing which specific content features drove each moderation decision

Auditability is a secondary benefit. The primary benefit in this validation/retry workflow is enabling the system to detect and feed back reasoning inconsistencies for self-correction.

WHERE THIS COMES FROM

Lesson 4.4: Validation, Retry, and Feedback Loops (detected\_patterns fields)

Lesson 4.4: Validation, Retry, and Feedback Loops (Self-correction flow)

Q07 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-002 · Multi-Agent Research System

SKIPPED

A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

- A** Split into two separate systems: one for simple queries and one for complex queries

Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

- B** Have the coordinator select which subagents to invoke per query, not always the full pipeline.

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests. (correct answer)

- C** Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline

Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.

- D** Add a caching layer in front of the pipeline so subagents with no work to do return a cached empty result immediately.

Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q08 · D3 3.5 ITERATIVE-REFINEMENT / EXAMPLES q-3-5-010 · Code Generation with Claude Code

SKIPPED

A data-transformation prompt keeps producing inconsistent output from prose instructions alone. Which refinement techniques should you apply? (Select 3)

- A** Collect every independent issue into one combined message to save round trips.

The guide batches issues into one message only when the fixes interact; independent problems are fixed sequentially.

- B** Rewrite the prose instructions as longer, more detailed paragraphs.

The problem is prose interpretation itself; the guide's answer is examples, not more prose.

**C** Write a test suite first, then iterate by sharing the failing tests.

Test-driven iteration turns each failure into precise, actionable feedback for the next pass. (correct answer)

**D** Provide two or three concrete input/output examples of the expected transformation.

Concrete examples are the guide's most effective way to communicate transformations that prose describes inconsistently. (correct answer)

**E** Add edge-case examples covering the inputs where the output has been most inconsistent.

Extending the example set to the specific inputs that drift is the documented refinement for inconsistent transformation output. It works for the same reason the base examples do: it replaces prose the model can interpret loosely with concrete pairs it cannot. (correct answer)

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Example-based communication)

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Lesson 3.5: Iterative Refinement Techniques (Batch vs sequential)

Q09 · D3 3.1 CLAUDE-MD-HIERARCHY / USER-LEVEL q-3-1-015 · Code Generation with Claude Code

SKIPPED

A developer has personal code formatting preferences (2-space indentation, trailing commas) that differ from some of the projects they work on. They want Claude Code to apply these preferences across all of their projects by default. Where should these preferences be configured?

**A** In `~/.claude/CLAUDE.md`, so they apply across all of the developer's projects

User-level CLAUDE.md (`~/.claude/CLAUDE.md`) applies to all of that developer's projects and is not shared with the team via version control, which is exactly right for personal preferences. Note that CLAUDE.md files are concatenated into context rather than strictly overriding one another, so a project's conflicting standard is not guaranteed to win; if a rule must always be enforced, put it in `settings.json` or a hook. (correct answer)

**B** In the project-level `.claude/CLAUDE.md` of each repository they work on

Project-level configuration is shared with the team via version control. Personal formatting preferences should not be imposed on the entire team. Additionally, this would require duplicating the preferences across every

project.

- C** In a `~/.claude/skills/` file that is invoked before each coding session

Skills are on-demand workflows invoked for specific tasks, not always-loaded configuration. Formatting preferences need to be applied automatically to every session without manual invocation.

- D** In a `.claude/rules/` file without YAML frontmatter so it loads for every session

`.claude/rules/` is project-scoped and version-controlled. Personal preferences should not be committed to project repositories where they would affect all team members.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (User-level CLAUDE.md)

Claude Code: Memory and CLAUDE.md ↗

Q10 · D4 4.1 SYSTEM-PROMPTS / KEYWORD-OVERLAP q-4-1-005

SKIPPED

A CI/CD system prompt defines two review categories with the instructions 'Check for security vulnerabilities in each function' and 'Check for performance issues in each loop'. The model frequently calls ``performance_check`` for security issues found inside loops, and ``security_check`` for performance issues in security-sensitive functions. What is the root cause and best fix?

- A** Force `tool_choice` to 'auto' and let the model determine the correct tool based on the content of each finding

`tool_choice` 'auto' is likely already the setting. The problem is that the model's tool selection is being misled by keyword overlap in the prompt, not by the `tool_choice` configuration.

- B** The model is confused because loops can have both security and performance issues; add a rule that security always takes priority over performance

Priority rules add complexity without fixing the root cause. The model is confusing tools because the instruction keywords overlap with both tool names and each other.

- C** Keyword overlap between the instruction phrasing and the tool names causes the confusion; rewrite them to use distinct, non-overlapping terms.

When instruction text like 'check for security in each function' overlaps with tool name patterns, the model

conflates instruction keywords with tool selection cues. Using distinct terminology for instructions versus tool names eliminates this cross-contamination. (correct answer)

- D Add more detailed tool descriptions explaining exactly when each tool should be called

More detailed descriptions help but do not fix the root cause if the instruction text still contains overlapping keywords that cue the wrong tool.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Keyword overlap)

Anthropic: Tool Use Documentation ↗

Q11 · D5 5.4 CODEBASE-EXPLORATION / SCRATCHPAD q-5-4-001 · Code Generation with Claude Code SKIPPED

A Claude Code agent is exploring an unfamiliar codebase. After investigating several modules, it starts referencing 'typical repository patterns' instead of the specific class names and dependency chains it discovered earlier. What is the most effective mitigation?

- A Increase the model's context window to accommodate more discovery output

Context degradation is not about running out of tokens — it is about the model losing grip on earlier findings as verbose output accumulates.

- B Have the agent maintain scratchpad files recording key findings and reference them for subsequent questions

Scratchpad files persist knowledge across context boundaries, directly counteracting context degradation by keeping critical discoveries accessible. (correct answer)

- C Pre-load the entire codebase structure into the initial context before exploration begins

This would consume context budget before exploration even starts and does not address degradation during the session.

- D Restart the session and ask the agent to explore more efficiently

Restarting loses all accumulated knowledge without addressing the underlying context degradation problem.

WHERE THIS COMES FROM

## Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

## Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files)

Q12 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-007 · Content Moderation and Classification System

SKIPPED

A moderation team introduces a severity scale ('low', 'medium', 'high', 'critical'). The same post (a user threatening to 'destroy' a competitor's product in a review) is classified as 'high' on some runs and 'low' on others. The prompt defines severities with phrases like 'high means the content is clearly harmful' and 'low means the content is mildly inappropriate'. What should they change?

- A Set the temperature to 0 to eliminate classification variance from one run of the pipeline to the next.

Lower temperature reduces randomness but does not fix ambiguous criteria. The model is inconsistent because the severity definitions are vague, not because of sampling variance.

- B Remove the severity scale and use binary classification (violation / not violation) to reduce inconsistency

Binary classification loses valuable granularity needed for routing decisions (auto-remove vs escalate vs warn). The fix is better severity criteria, not removing severity entirely.

- C Replace the prose severity descriptions with a concrete example per level as a calibration anchor.

Concrete examples anchor each severity level to specific, observable patterns. When the model sees 'destroy a competitor's product' in a review context, it can match against the 'low severity — figurative language' example rather than interpreting 'clearly harmful' differently each run. (correct answer)

- D Run the classification three times over and take the majority vote to settle the severity for each item.

Majority voting averages out noise but triples cost and latency without fixing the root cause. With vague criteria, the model may consistently misclassify in the same direction, making voting ineffective.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q13 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-AMBIGUITY q-4-2-007

SKIPPED

An invoice-extraction pipeline mislabels fields on documents where the correct interpretation is ambiguous. Which few-shot practices should you apply? (Select 2)

- A Create two to four targeted examples for the ambiguous cases, showing why one interpretation is chosen.

Targeted examples that demonstrate the reasoning are the guide's specific remedy for ambiguous cases. (correct answer)

- B Replace the examples with a longer natural-language description of each field.

Descriptions alone are what produced the inconsistency; examples communicate the expected mapping more reliably than more prose.

- C Use examples showing correct extraction from varied document structures to fix empty or null extraction of required fields.

Varied-structure examples fix empty or null extraction of fields that are present, which is the few-shot remedy for unreliable extraction; fabrication of absent data is handled by nullable schema fields instead. (correct answer)

- D Add an example for every document variant seen in production so far.

Volume is not the fix; a handful of targeted examples for the ambiguity beats exhaustive coverage and its context cost.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot construction)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q14 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-001 · Multi-Agent Research System

SKIPPED

A developer resumes a Claude Code session after modifying 3 files in a 50-file codebase. The agent gives contradictory advice about the modified files because it is reasoning from stale tool results. What is the correct approach?

- A** Start a fresh session with an injected summary of prior findings, noting the 3 changed files for targeted re-analysis.

A fresh session with summary injection avoids stale tool results. Specifying the 3 changed files enables targeted re-analysis without wasting time on unchanged files. (correct answer)

- B** Start a completely new session and have the agent re-analyse all 50 files from scratch so nothing stale survives into the new context.

Re-analysing all 50 files is wasteful when only 3 changed. A fresh session with summary injection plus targeted re-reading of the 3 changed files is more efficient.

- C** Use `fork_session` to branch the session so the new branch picks up the 3 changed files while the original keeps the earlier state.

`fork_session` creates an independent exploration branch from a shared baseline, and that baseline still carries the stale results. It is for divergent approaches, not for updating stale context after file changes.

- D** Resume the session and ask the agent to re-read only the 3 modified files, overwriting the stale results already in the context window.

Resuming preserves the stale tool results in context, and a later read does not overwrite an earlier one. The agent may still reason from outdated information for other decisions. A fresh start with summary injection is more reliable.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Targeted re-analysis)

Anthropic: Claude Code Documentation ↗

Q15 · D1 1.1 AGENTIC-LOOPS / MODEL-DRIVEN q-1-1-008 · Content Moderation and Classification System

SKIPPED

The moderation system's agentic loop uses a hardcoded decision tree: if the `classify_content` tool returns 'hate\_speech', always call `escalate_to_human`; if it returns 'spam', always call `auto_remove`. During testing, the team discovers that satirical posts criticising hate speech are being auto-escalated, and sophisticated spam disguised as legitimate marketing slips through. What is the architectural problem?

- A Route to human review every case the decision tree cannot classify cleanly, so satire and disguised spam are both caught by a person.

Routing everything ambiguous to humans defeats the purpose of automated moderation. The model can reason about context effectively when given the opportunity; the hardcoded tree is what prevents it.

- B Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.

Hardcoded decision trees treat classification labels as absolute when real content is nuanced. Letting Claude reason about context (satire vs genuine hate, sophisticated spam patterns) produces better moderation decisions. The agentic loop should let the model decide, not map labels to fixed actions. (correct answer)

- C The `classify_content` tool needs more granular category labels so it can tell satire criticising hate speech apart from genuine hate speech.

More granular labels do not address the core issue. A hardcoded decision tree cannot adapt to nuance, such as satire versus genuine hate, no matter how fine the labels are. The fix is model-driven reasoning.

- D Add a confidence threshold so only high-confidence classifications trigger automatic actions

Confidence thresholds reduce false positives but the fundamental problem remains: a hardcoded tree cannot handle contextual nuance. Low-confidence cases still need model-driven reasoning, not just deferral.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Model-driven decision-making)

Lesson 1.1: Agentic Loops (Anti-patterns)

Q16 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-005 · Multi-Agent Research System

SKIPPED

A research coordinator spawns 4 subagents to investigate different aspects of a market analysis. Agent A fails with an API error, Agent B returns partial results (3 of 5 requested data points), and Agents C and D return complete results. The final report is due in 2 hours. What should the coordinator do?

- A Retry Agent A, incorporate Agent B's partial results, flag B's missing data points as gaps, and synthesise from all available results.

The coordinator should maximise data utilisation: retry the failed agent (A), incorporate partial results from B while flagging the 2 missing data points as known gaps, and synthesise all available results from A (after retry), B (partial), C, and D. Explicit gap annotations ensure the report is transparent about its completeness. (correct answer)

- B** Proceed with only Agents C and D's results and produce the report without the data from Agents A and B

This unnecessarily discards Agent B's partial results and abandons Agent A's scope entirely. A thorough coordinator should attempt recovery while incorporating all available data.

- C** Retry Agent A, discard Agent B's partial results as unreliable, and wait for both agents to return fully complete data before writing the report.

Discarding Agent B's partial results wastes valid data, and waiting for full completion ignores the two-hour deadline. The coordinator should use what is available while attempting to fill gaps.

- D** Terminate all agents and restart the entire investigation from scratch with a simplified scope

Restarting discards the complete results from Agents C and D and the partial results from Agent B. This wastes time and compute when most of the investigation has already succeeded.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Lesson 1.7: Session State and Resumption (Recovering from partial results)

**Q17** · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / STRUCTURED-METADATA q-1-3-001 · Multi-Agent Research System

SKIPPED

A synthesis agent produces a report with several claims that have no source attribution. The web search and document analysis subagents are working correctly and returning well-sourced results. What is the most likely root cause?

- A** The web search subagent returns prose summaries rather than a structured citation format the synthesis agent can turn into attributions.

The web search subagent is working correctly and returns well-sourced results. The issue is how the coordinator passes those results to the synthesis agent, not the shape the subagent emits.

- B The synthesis agent's system prompt does not instruct it to include citations, so it drops source attribution even when given well-sourced input.

Even with a citation instruction, the synthesis agent cannot attribute claims to sources that were never passed to it. The lesson is explicit: do not blame the synthesis prompt; the root cause is the coordinator dropping structured metadata.

- C The synthesis agent has no direct web search access, so it cannot re-fetch the sources behind the claims it has been asked to write up.

Giving the synthesis agent web search tools violates the principle of scoped tool access, and re-fetching is not the fix. The coordinator is dropping attribution metadata it already holds.

- D The coordinator passes content to the synthesis agent stripped of structured metadata like source URLs and document names.

Context passing must include structured data that separates content from metadata. Without source URLs and document names, the synthesis agent has no attribution information to include. (correct answer)

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Structured metadata format)

Lesson 1.3: Subagent Invocation and Context Passing (Context passing)

Q18 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-001 · Multi-Agent Research System

SKIPPED

A multi-agent research system produces a synthesis report on market trends. Two credible sources report different growth rates for the same sector: Source A reports 12% growth (2023 data) and Source B reports 8% growth (2024 data). The synthesis agent currently selects the more recent value. What is the correct approach?

- A Average the two values and report 10% growth, with a note recording the variance between sources.

Averaging conflicting statistics is mathematically misleading and destroys the temporal context that explains the difference.

- B** Annotate both values with source and publication dates, letting the consumer interpret them.

This preserves provenance, temporal context, and both data points. The consumer can see that different dates explain different numbers. (correct answer)

- C** Always use the most recent of the two sources, on the grounds that it reflects the latest data.

This silently discards valid historical context. The difference may reflect an actual trend, not a contradiction.

- D** Flag the conflict and escalate to a human researcher for resolution before including in the report

This is unnecessary — the values are not contradictory, they reflect different time periods. Proper annotation resolves the ambiguity.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Temporal awareness)

Q19 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-011 ·

Enterprise Data Platform with Federated Queries

SKIPPED

Your federated query platform exposes several similar search tools, and Claude keeps routing requests to the wrong one. Which changes improve tool selection reliability? (Select 3)

- A** Merge the similar tools into one generic tool with a mode parameter.

The guide moves in the opposite direction: split generic tools into purpose-specific tools with defined contracts.

- B** Rewrite each description to state the tool's purpose, inputs and outputs, and when to prefer it.

Descriptions are the primary signal Claude uses to choose tools; differentiating them is the first fix for misrouting. (correct answer)

- C** Trim every description to one short sentence so the model relies on tool names alone.

Minimal descriptions cause unreliable selection among similar tools; names alone cannot carry boundary information.

- D** Check the system prompt for keyword-sensitive instructions that create unintended associations with particular tools.

System prompt wording can override well-written descriptions, so it belongs in the same review. (correct answer)

- E** Rename tools whose names overlap so each name reflects a distinct function.

The guide's example renames `analyze_content` to `extract_web_results`, removing the functional overlap that caused misrouting. (correct answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Misrouting)

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

Anthropic: Tool Use Documentation ↗

Q20 · D4 4.6 MULTI-PASS-REVIEW / SELF-REVIEW-BIAS q-4-6-005 ·

Content Moderation and Classification System

SKIPPED

The moderation team asks a single Claude session to classify a post, then immediately asks the same session to independently review its own classification for quality assurance. The 'review' agrees with the original classification 98% of the time, including cases that human auditors later identify as errors. Why is this self-review ineffective?

- A** The model's temperature is too low, producing deterministic agreement — increase temperature for the review pass

Temperature affects sampling randomness, not reasoning independence. Higher temperature may occasionally produce different outputs but does not create genuine independent review. The model is still anchored to its own prior reasoning in the same session.

- B** The same session retains the model's reasoning, so it stays anchored to its classification; use a fresh independent instance to review.

Self-review within the same session is fundamentally limited because the model's conversation context includes the original reasoning. It will naturally be anchored to its prior conclusions. An independent instance with no access to the original decision evaluates the content fresh, providing genuine quality assurance. (correct answer)

- C** Self-review is effective but the 98% agreement rate simply reflects high initial accuracy — the 2% disagreement is the expected error rate

Human auditors found errors in cases where the self-review agreed, proving the 98% agreement does not reflect accuracy. The self-review is confirming errors, not catching them, due to same-session reasoning bias.

- D** The model needs a stronger review prompt with explicit instructions to look for errors in the original classification

Stronger instructions do not overcome the fundamental limitation. The same session retains the reasoning context from the original classification, biasing the review toward agreement.

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

Q21 · D4 4.5 BATCH-PROCESSING / BATCH-VS-REALTIME q-4-5-001

SKIPPED

Your manager proposes switching both your blocking pre-merge code review and your overnight technical debt report to the Message Batches API for 50% cost savings. How should you evaluate this proposal?

- A** Switch both to batch with a timeout fallback to real-time if batches take too long

This adds unnecessary complexity. Match each API to its latency requirements: synchronous for blocking, batch for tolerant.

- B** Switch both to batch processing with status polling to check for completion

Batch processing has no guaranteed latency SLA. Polling does not make it suitable for blocking pre-merge checks.

- c Keep real-time calls for both to avoid batch result ordering issues

Batch results can be correlated using custom\_id fields. Ordering is not the issue — latency requirements are.

- d Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks

Pre-merge checks are blocking workflows where developers wait. The batch API's 24-hour window makes it unsuitable. Technical debt reports are overnight and latency-tolerant. (correct answer)

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Batches API facts)

Lesson 4.5: Batch Processing and Prompt Optimisation (SLA implications)

Anthropic: Message Batches API ↗

Q22 · D3 3.1 CLAUDE-MD-HIERARCHY / HIERARCHY q-3-1-001 · Code Generation with Claude Code

SKIPPED

Developer A's Claude Code follows the team's API naming conventions perfectly. Developer B, who joined last week, gets inconsistent naming from Claude Code. Both are working on the same repo, same branch. What is the most likely root cause?

- A The conventions are in a .claude/rules/ file that Developer B's editor is not configured to load, so they never reach his session

Rule files in .claude/rules/ are version-controlled and loaded by Claude Code from the repository, not by the editor. On the same repo and branch, both developers get identical rule loading, so an 'editor not loading rules' story cannot explain the difference.

- B The API naming conventions live in Developer A's user-level CLAUDE.md, not the project-level configuration

User-level CLAUDE.md is not version-controlled or shared via git. Developer A has the instructions locally; Developer B, having just joined, does not. Moving instructions to project-level config (.claude/CLAUDE.md) would fix this. (correct answer)

- c Developer B has not run /memory to load the configuration files into the session.

/memory is a debugging command to verify what is loaded, not a command to trigger loading. Configuration

files load automatically.

- D Developer B needs to install an MCP server before the naming convention rules become available to Claude Code.

MCP servers provide tool integrations, not CLAUDE.md configuration. Naming conventions belong in CLAUDE.md files.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Three-level hierarchy)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (New-team-member scenario)

Claude Code: Memory and CLAUDE.md ↗

Q23 · D3 3.5 ITERATIVE-REFINEMENT / EXAMPLES q-3-5-001 · Code Generation with Claude Code

SKIPPED

A developer describes a code transformation in prose. Claude Code interprets it differently each time, producing inconsistent results. What technique should the developer try first?

- A Use the interview pattern to have Claude ask clarifying questions before implementing

The interview pattern is best for unfamiliar domains where the developer might miss considerations. When the developer knows the exact transformation but the model interprets it inconsistently, examples are the direct fix.

- B Write a test suite first and iterate by sharing test failures

Test-driven iteration is effective but is a more heavyweight approach. Concrete examples are the faster first step when the issue is inconsistent interpretation of a known transformation.

- C Rewrite the prose description with more precise language and technical terminology

More precise prose still relies on the model's interpretation of natural language. If interpretation is inconsistent, more prose will not solve the root cause.

- D Provide 2-3 concrete input/output examples showing the exact before and after transformation

Concrete examples eliminate interpretation ambiguity. The model generalises from examples more reliably than from descriptions. This is the documented first-line technique for inconsistent interpretation. (correct answer)

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Example-based communication)

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Q24 · D4 4.3 STRUCTURED-OUTPUT / ENUM-FIELDS q-4-3-004 · Content Moderation and Classification System

SKIPPED

The moderation system's classification schema has a 'category' field defined as a free-text string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate\_speech' — all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?

- A Change 'category' from free-text to an enum with values like 'hate\_speech', 'spam', and 'harassment', plus an 'other' option.

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned. (correct answer)

- B Add few-shot examples showing the correct category formatting for each type

Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

- C Add a post-processing normalisation step that maps all variations to canonical category names

Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream.

- D Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.

Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.

WHERE THIS COMES FROM

Lesson 4.3: Structured Output with Tool Use (Enum schema)

Q25 · D3 3.6 CI/CD-INTEGRATION / SESSION-ISOLATION q-3-6-002

SKIPPED

A CI/CD pipeline uses Claude Code to generate tests in one step and then review the same code in a subsequent step. The team notices that the review step rarely finds issues with the generated tests, but human reviewers consistently find problems. Both CI steps use the `-p` flag correctly. What is the most likely cause and fix?

- A The review step runs the same model tier as generation, which is not capable enough to critique its own output; upgrade the review step to a more powerful model

Model capability is not the bottleneck. A more powerful model still inherits self-review bias when it runs in the generating session's context, because it retains the reasoning it used to justify the code. The pattern here (a review that passes work human reviewers reject) points to shared reasoning context, which is fixed by running the review as an independent session with no shared context, not by upgrading the model.

- B The CLAUDE.md file lacks testing standards and review criteria, so the review has no project-specific context

While documenting testing standards in CLAUDE.md is important, it would affect the quality of both generation and review. The specific pattern here — review missing issues that humans catch — points to self-review bias, not missing context.

- C The review step needs `--output-format json` to produce structured findings that can be parsed programmatically

The output format affects how findings are reported, not whether issues are detected. The problem is that the review misses issues entirely, not that findings are in the wrong format.

- D The generation and review steps must run as independent sessions with no shared context, so the review instance approaches the code without the original reasoning bias

The same Claude session (or a session with carried-over context) that generated code is less effective at reviewing its own changes because it retains the reasoning context from generation. An independent review instance without that context is more likely to catch issues, matching what human reviewers find. Session context isolation for independent reviews is a key CI integration concept. (correct answer)

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Session context isolation)

Q26 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-002 · Multi-Agent Research System

SKIPPED

A multi-agent research system has a coordinator that spawns a web search subagent and a document analysis subagent sequentially across separate API turns. The web search completes in 8 seconds and the document analysis completes in 12 seconds, giving a total latency of 20 seconds. The two subagents are investigating independent topics and do not depend on each other's results. How should the architect reduce this latency?

- A Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds

When subagents investigate independent topics, the coordinator can emit multiple Task tool calls in a single response. Both subagents run concurrently, so total latency is determined by the slower one (12 seconds) rather than the sum (20 seconds). (correct answer)

- B Use `fork_session` to split the coordinator into two parallel branches, one for each subagent

`fork_session` is for divergent exploration of the same problem from a shared baseline, not for parallel subagent invocation. Emitting multiple Task tool calls in a single coordinator response is the correct mechanism for spawning concurrent subagents.

- C Merge the web search and document analysis into a single subagent to reduce coordination overhead

Merging agents violates the principle of specialised subagents with scoped tool access. It also does not reduce total processing time — the merged agent still needs to do both tasks, and loses the benefit of parallel execution.

- D Allow the web search subagent to directly invoke the document analysis subagent after it finishes, removing the coordinator round-trip

Direct subagent communication violates the hub-and-spoke architecture where all communication flows through the coordinator. This also would not achieve parallel execution — it would still be sequential.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Q27 · D1 1.2 ORCHESTRATION-PATTERNS / DECOMPOSITION q-1-2-001 · Multi-Agent Research System

SKIPPED

A multi-agent research system produces a report on 'renewable energy technologies' that only covers solar and wind, missing geothermal, tidal, biomass, and nuclear fusion. Each subagent produced thorough coverage of its assigned topic. Where is the root cause?

- A The web search subagent scoped its queries too narrowly, so geothermal, tidal, biomass and fusion never surfaced in the results it returned.

The web search subagent researched what it was assigned, and it returned thorough coverage of that assignment. The issue is what it was assigned, not how narrowly it searched.

- B The coordinator's task decomposition assigned only solar and wind, omitting the other renewable categories.

The coordinator is responsible for decomposing the broad topic into subtopics. If it only assigns solar and wind, no downstream agent can cover the missing categories. Trace failures to their origin. (correct answer)

- C The document analysis subagent lacked access to sources on geothermal, tidal, biomass, and fusion, so those categories never entered the report.

Source availability is not the root cause. The coordinator never asked any agent to research geothermal, tidal, biomass, or fusion, so no fix to source access would help.

- D The synthesis subagent failed to audit its inputs for coverage gaps, so it wrote up solar and wind as though they were the whole field.

The synthesis agent can only work with the research it receives. A coverage audit would flag the omission, but it cannot synthesise topics that were never researched.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Narrow decomposition failure)

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q28 · D5 5.4 CODEBASE-EXPLORATION / COMPACT q-5-4-006 · Enterprise Data Platform with Federated Queries

SKIPPED

A developer has been using the data platform agent in a single extended session for two hours, running complex federated queries, debugging SQL syntax, and iterating on report formatting. The

agent now produces noticeably worse SQL and occasionally references table schemas from earlier queries that have since been corrected. What is the most effective intervention?

- A Start a completely new session and re-establish all context from scratch.

A fresh session discards all accumulated context, including valuable recent work. `/compact` summarises the existing conversation while preserving the gist of recent work, which is less disruptive than starting over.

- B Increase the model's temperature parameter to improve creativity and reduce fixation on stale patterns.

Temperature affects output randomness, not context quality. The problem is stale information in the context window polluting the agent's reasoning, not a lack of creative variation.

- C Add a system prompt instruction telling the agent to ignore all schema information from more than 30 minutes ago.

The agent has no reliable concept of message timestamps within the context window. Time-based instructions are unenforceable. The structural fix is to compress or remove stale context entries.

- D Use `/compact`, optionally with focus instructions, to summarise the conversation and free context budget.

Extended sessions cause context degradation as stale information (outdated schemas, abandoned approaches) competes with current context. `/compact` summarises the conversation history into a compressed form, freeing context budget. The optional `[instructions]` argument lets you bias the summary (for example, 'preserve the corrected schema for the orders table'). It is the documented mitigation for context degradation in long sessions, and the least disruptive option of the four. (correct answer)

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (`/compact`)

Q29 · D3 3.3 PATH-SPECIFIC-RULES / PATHS-FRONTMATTER q-3-3-008 · Code Generation with Claude Code

SKIPPED

A large TypeScript monorepo has a root `CLAUDE.md` with universal coding standards and a `.claude/rules/` directory with topic-specific rule files. A developer notices that conventions from `testing.md` in `.claude/rules/` are loading even when editing API handler files, consuming unnecessary tokens. The `testing.md` file has no YAML frontmatter. What should the developer do to fix this?

- A Use an `@./claude/rules/testing.md` line in the root `CLAUDE.md` so the file only loads when the import is reached

`@ path` imports load eagerly when the `CLAUDE.md` is loaded — the imported file's content is inlined at that moment, regardless of which file the developer is editing. They are a source-organisation mechanism, not a conditional-loading mechanism. Path-scoped frontmatter in `.claude/rules/` is the only way to make rule files load only for matching paths.

- B Add the testing conventions to the root `CLAUDE.md` instead, since rules files cannot be path-scoped

Rules files absolutely support path scoping via YAML frontmatter with a `paths` field. Moving conventions to root `CLAUDE.md` would make the token efficiency problem worse, not better, because root `CLAUDE.md` loads for every session.

- C Move `testing.md` out of `.claude/rules/` and into a directory-level `CLAUDE.md` inside the test folder

Test files are co-located with source files throughout 200+ packages in this monorepo. A directory-level `CLAUDE.md` in a single test folder would not cover test files in other directories, and creating one in every directory would be unmaintainable.

- D Add YAML frontmatter with a `paths` field of test-file globs to `testing.md` so it loads only for test files

Without YAML frontmatter, rule files in `.claude/rules/` load for all sessions. Adding a `paths` field with glob patterns restricts loading to sessions where the developer is editing matching files. This is the correct approach for token-efficient, conditional convention loading. (correct answer)

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.1: `CLAUDE.md` Hierarchy and Scoping (`.claude/rules/` directory)

Q30 · D5 5.3 ERROR-PROPAGATION / STRUCTURED-ERROR-CONTEXT q-5-3-001 · Multi-Agent Research System

SKIPPED

A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

- A Propagate the timeout exception to a top-level handler that terminates the entire research workflow at once, discarding everything.

Workflow termination wastes partial results from other subagents that may have completed successfully.

- B Catch the timeout and return an empty result set marked as successful, so the rest of the workflow carries on regardless of the failure.

Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.

- C Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results. (correct answer)

- D Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted

The generic status hides valuable context from the coordinator, preventing informed recovery decisions.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

Q31 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-002

SKIPPED

Your Claude Code review prompt for the pipeline's polyglot codebase classifies severity using prose descriptions like 'critical means the code is dangerous' and 'minor means the code is slightly suboptimal'. Developers complain that identical code patterns receive different severity ratings across runs. What is the most effective improvement?

- A Add a confidence threshold so only findings above 90% confidence are reported, filtering out uncertain severity ratings

Self-reported confidence scores are poorly calibrated. Confidence-based filtering does not fix the root cause, which is ambiguous severity criteria. Explicit code examples are the correct fix.

- B** Replace prose severity descriptions with concrete TypeScript code examples for each severity level.

Severity calibration with concrete code examples produces consistent classification across invocations. Prose descriptions are inherently ambiguous and leave the model to interpret severity differently each time. (correct answer)

- C** Add a second model pass that re-evaluates each finding's severity to catch inconsistencies

A second pass using the same vague prose criteria will produce the same inconsistency. The criteria themselves must be improved with concrete code examples before adding verification layers.

- D** Lower the model temperature to 0 to ensure deterministic severity ratings

While lower temperature reduces randomness, it does not address the fundamental problem: the model has no concrete reference for what each severity level looks like. Code examples provide that reference.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q32 · D2 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS q-2-4-006 ·

Enterprise Data Platform with Federated Queries

SKIPPED

An MCP server exposes a `query_database` tool for Snowflake. Agents ignore it and run SQL via the built-in Bash tool against the Snowflake CLI, even though the MCP tool returns structured results with column types and pagination that Bash does not. What is the most likely cause and fix?

- A** Add a system prompt instruction telling the agent to always use `query_database` instead of Bash for SQL queries.

Whilst a system prompt instruction could work as a short-term fix, it is brittle and does not scale. The root cause is the sparse description. Enhancing the description is the sustainable fix that works across all agents and contexts.

- B** The MCP server is not properly connected. Restart the MCP server and verify the connection with a test query.

If the server were disconnected, the tool would not appear in the available tools list at all. The agent is choosing

not to use it, which is a description problem, not a connectivity problem.

- c Disable the Bash tool entirely so the agent is forced to use the MCP tool for all operations.

Disabling Bash removes a critical general-purpose tool needed for many operations beyond database queries. The fix should make the MCP tool more attractive for its specific use case, not remove other tools.

- d Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.

When MCP tool descriptions are sparse, agents default to the familiar Bash tool. Enhancing the description to explain the structured results, column types, and pagination gives the agent enough to prefer the MCP tool for database queries. (correct answer)

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q33 · D2 2.5 BUILT-IN-TOOLS / GREP-VS-GLOB q-2-5-008 · Enterprise Data Platform with Federated Queries

SKIPPED

A developer is investigating a data inconsistency in the platform codebase. They need to find all files that reference a specific Snowflake table name 'fact\_revenue\_daily' and then check the directory structure for related migration files. Which tool sequence is correct?

- A Read all SQL files in the project to search for the table name, then Read all migration files.

Reading all files upfront is a context-budget anti-pattern. It wastes tokens on irrelevant files and is exactly the approach the exam penalises.

- B Grep for 'fact\_revenue\_daily' to find all content references, then Glob for '\*\*/migrations/\*.sql' to find migration files by path pattern.

Grep searches file contents — correct for finding code that references the table name. Glob matches file paths — correct for finding migration files by directory pattern. This is the optimal two-step sequence using each tool for its strength. (correct answer)

- c Grep for 'fact\_revenue\_daily' for both steps — first to find references, then to find migration files containing the table.

Using Grep for both steps would find migration files that mention the table, but would miss migration files related to the table that do not contain its name (e.g. schema migrations, index changes). Glob is the correct tool for path-based file discovery.

- d Glob for '\*\*/\*fact\_revenue\_daily\*' to find references, then Glob for '\*\*/migrations/\*\*' to find migration files.

Glob matches file paths by name, not file contents. It would find files named after the table (unlikely) but miss all files that reference the table name in their code.

WHERE THIS COMES FROM

### Lesson 2.5: Built-in Tools (Grep vs Glob)

Q34 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / CONFIDENCE-ROUTING q-1-4-007 ·

Content Moderation and Classification System

SKIPPED

The moderation system auto-removes posts classified as policy violations. Production logs reveal that 3% of auto-removed posts were legitimate content (false positives), generating user complaints and eroding platform trust. The team considers two approaches: (A) escalate all borderline cases to human reviewers, or (B) add a confidence threshold where only high-confidence violations are auto-removed and everything else goes to human review. Which approach is better and why?

- A Approach B: auto-remove only high-confidence violations, escalate uncertain cases to human review, and auto-approve high-confidence safe content.

Confidence-based routing creates a tiered system: high-confidence violations are actioned immediately, high-confidence safe content passes through, and uncertain cases receive human review. This balances speed, accuracy, and reviewer workload while directly addressing the false positive problem. (correct answer)

- B Approach A: escalate every borderline case, because human reviewers are more accurate than the classifier and a 3% false-positive rate is a trust problem.

Escalating all borderline cases overwhelms the human review queue without leveraging the system's ability to handle clear-cut cases autonomously. The goal is to route intelligently, not to defer everything.

- C** Neither — instead lower the classification threshold so fewer posts are flagged as violations

Lowering the threshold reduces false positives but increases false negatives (genuine violations going undetected). This trades one problem for another rather than adding an appropriate escalation path.

- D** Approach B, but apply a single fixed 95% confidence threshold to every category regardless of how costly a false positive is for that content type.

A single fixed threshold across all categories ignores that violation types have different risk profiles. The threshold should be calibrated per category based on the cost of false positives versus false negatives.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Confidence-based escalation)

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum)

Q35 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-003 · Content Moderation and Classification System

SKIPPED

The moderation system validates that every classification includes a 'reasoning' field explaining the decision. When validation fails (empty reasoning), the system retries. For most posts, the retry succeeds after feeding back the error 'reasoning field was empty — provide a brief justification for the classification.' However, for posts in unfamiliar languages, retries consistently fail. What should the system do?

- A** Remove the reasoning validation requirement for posts in unfamiliar languages

Removing the validation requirement means these posts would receive classifications without any justification, which undermines auditability. The correct approach is human review, not lower standards.

- B** Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.

The retry boundary distinguishes between fixable format issues and unfixable capability gaps. Empty reasoning on analysable posts is a format error that retries with error feedback can fix. Unfamiliar languages represent absent capability — no amount of retrying will create language understanding. Route these to human review.

(correct answer)

- C** Increase the retry count from 1 to 5 for unfamiliar language posts since the model may succeed with more attempts

More retries are ineffective when the model lacks the capability to analyse content in that language. The failure is not a transient format issue — the model genuinely cannot provide reasoning for content it cannot understand.

- D** Add the unfamiliar language to the system prompt as a supported language to encourage the model to try harder

Listing an unsupported language in the prompt does not give the model language understanding. This may actually worsen the problem by encouraging fabricated analysis of content the model cannot read.

WHERE THIS COMES FROM

#### Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

**Q36** · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-008 ·

Enterprise Data Platform with Federated Queries

SKIPPED

The data platform's `fetch_api` tool calls a third-party pricing API. When the API key has expired, the tool returns `{ "data": [], "status": "ok" }` instead of signalling an authentication failure. The agent tells the user 'No pricing data is available for that product' and moves on. What is the root cause?

- A** The agent should be instructed via the system prompt to treat empty arrays as errors and retry with exponential backoff.

Not all empty arrays are errors — a product with no pricing data is a valid empty result. The problem is that the tool masks an authentication failure as a successful empty response. System prompt instructions cannot reliably distinguish the two.

- B** The third-party API is poorly designed. The platform team should switch to a different pricing provider that returns proper HTTP error codes.

Whilst the upstream API behaviour is unhelpful, the platform team controls the MCP tool layer. The tool should detect the authentication failure (even from the upstream response) and translate it into a structured MCP error rather than passing through the misleading response.

- C** The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.

The tool returns the same structure for two fundamentally different outcomes: 'I could not access the data' versus 'I accessed the data and found nothing.' The agent cannot distinguish these and accepts the empty result at face value. The tool must return a structured error with the MCP `isError` flag for authentication failures.

(correct answer)

- D** The agent's context window is too small to hold the full API response, so it receives a truncated version that appears empty.

Context window truncation would not produce a well-formed `{ "data": [], "status": "ok" }` response. The problem is semantic — the tool is returning a misleading success status for an authentication failure.

WHERE THIS COMES FROM

## Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q37 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / ROLE-SCOPING q-2-3-007 ·

Enterprise Data Platform with Federated Queries

SKIPPED

An enterprise data platform agent has 22 tools available, including query tools for Snowflake, PostgreSQL, and third-party APIs, plus data transformation, visualisation, export, and administrative tools. The team notices the agent frequently selects the wrong query tool — choosing the Snowflake tool for data that lives in PostgreSQL. What is the most effective architectural fix?

- A** Improve the descriptions of all 22 tools to include explicit boundaries and data source mappings.

Task Statement 2.1 does teach descriptions as the first fix for misrouting, which is why this option is tempting, but that rule assumes the agent has a workable number of tools and simply cannot tell two of them apart. Here there are 22, well past the 4-5 range where selection stays reliable. Decision complexity degrades selection regardless of how well each description is written, so rewriting all 22 treats the symptom and leaves the cause.

- B** Split the agent into role-specific agents — a query agent with 4-5 data source tools, a transformation agent, and an export agent — each scoped to its specialisation.

Tool overload at 22 tools degrades selection reliability. Splitting into role-specific agents with 4-5 tools each reduces decision complexity and scopes each agent to its specialisation. The query agent can focus on selecting the right data source without being distracted by transformation or export tools. (correct answer)

- c Consolidate all query tools into a single `universal_query` tool that internally routes to the correct data source based on the query syntax.

A universal query tool hides the data source distinction from the agent. When the agent needs to reason about which data source contains the relevant data, it loses the ability to make that decision explicitly.

- d Add a pre-processing step that analyses the user query to determine which data source is needed, then use `tool_choice` forced selection to call the correct query tool.

Forced tool selection based on a pre-processing step is over-engineered and brittle. It bypasses the LLM's ability to reason about which tool to use and breaks when queries span multiple data sources.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Role-specific scoping)

Lesson 2.3: Tool Distribution and Tool Choice (Tool overload)

Q38 · D3 3.1 CLAUDE-MD-HIERARCHY / SPLITTING-RULES q-3-1-013 · Code Generation with Claude Code

SKIPPED

A growing project's `CLAUDE.md` has reached 800 lines covering API design rules, testing conventions, deployment procedures, and frontend style guides. Most of those topics only matter when working in the matching part of the codebase (API rules in `/api`, frontend rules in `/web`, etc.). Developers report attention dilution: Claude Code is increasingly inconsistent on specific conventions. What is the recommended fix to reduce the per-session token load while preserving coverage?

- A Split `CLAUDE.md` into per-topic files referenced with `@` path imports so each topic lives in its own file

`@` path imports load eagerly — Claude reads the imported file inline at load time, exactly as if its content were pasted into `CLAUDE.md`. The source files become smaller, but the loaded context stays the same size. This improves source-file readability and authoring ergonomics, but it does not solve the attention-dilution problem the question is asking about.

- B Move each topic's sections into a path-scoped `.claude/rules/` file that loads only in the matching area

Path-scoped `.claude/rules/` files are the documented way to reduce per-session context for guidance that only applies to part of the codebase. With frontmatter like ``paths: ["api/**"]``, the API rules only load when working in

/api; the frontend rules only load when working in /web; and so on. CLAUDE.md keeps only the genuinely cross-cutting standards. This actually shrinks the loaded context — unlike @ imports, which load eagerly. (correct answer)

- C** Duplicate the CLAUDE.md content across directory-level CLAUDE.md files in each subdirectory

Directory-level CLAUDE.md files load \*in addition to\* the project root file, not instead of it. Duplicating content would make the problem worse and create a maintenance nightmare. Use path-scoped .claude/rules/ files instead.

- D** Claude Code has a hard limit of 500 lines for CLAUDE.md files; trim the file to fit within the limit

There is no documented hard line limit for CLAUDE.md files. The issue is token economy and attention dilution at load time, not a technical file-size constraint.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Modular Organisation with @ path imports — eager-load warning)

Lesson 3.3: Path-scoped rules in .claude/rules/

Q39 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-007 ·

Enterprise Data Platform with Federated Queries

SKIPPED

An enterprise data platform team is evaluating how to integrate with their existing PostgreSQL database, Slack workspace, and a custom internal approval workflow. A developer proposes building three custom MCP servers. What is the correct approach?

- A** Use community MCP servers for PostgreSQL and Slack, and build a custom server only for the internal approval workflow that has no community equivalent.

Community servers should be evaluated first for standard integrations. PostgreSQL and Slack have well-maintained community MCP servers. The internal approval workflow is team-specific with no community equivalent, making it the only justified custom build. (correct answer)

- B** Use community MCP servers for all three integrations, adapting the internal approval workflow to fit an existing community server's interface.

Forcing a custom internal workflow into a community server's interface leads to awkward abstractions and

missing functionality. The community-first principle applies to standard integrations, not to genuinely unique team-specific workflows.

- C** Build all three custom MCP servers to ensure tight integration with the team's specific requirements and coding standards.

Building custom servers for PostgreSQL and Slack is unnecessary when well-maintained community servers already exist for these standard integrations. Custom builds should be reserved for genuinely unique workflows.

- D** Skip MCP entirely and use direct API calls from Bash for all three integrations to avoid the overhead of running MCP servers.

Direct API calls bypass the MCP tool interface, losing the benefits of structured tool descriptions, standardised error handling, and agent-native integration. MCP servers provide a consistent interface that agents understand natively.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q40 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CONTEXT q-2-2-011 · Multi-Agent Research System

SKIPPED

A research subagent's database query fails partway through an investigation. Which pieces of information belong in the structured error context it returns to the coordinator? (Select 3)

- A** The failure type, such as transient, validation, or permission.

Categorising the failure lets the coordinator make the right retry-or-reroute decision. (correct answer)

- B** Partial results gathered before the failure.

Partial results preserve completed work so the coordinator does not re-run what already succeeded. (correct answer)

- C** What was attempted, including the query that failed.

The attempted query gives the coordinator enough detail to retry intelligently or reformulate the request. (correct answer)

- D No report at all: the subagent should retry silently until the query succeeds.

Silent retries hide failures from the coordinator, which owns error handling and needs observability into what went wrong.

- E A plain 'no results found' message in place of the error.

Reporting an access failure as an empty result conflates two different situations and hides the need for a retry decision.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Four error categories)

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

MCP: Tools Specification ↗

Q41 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / TOOL-RESULT-TRIMMING q-5-1-005 ·

Enterprise Data Platform with Federated Queries

SKIPPED

An analytics agent queries Snowflake and receives 40+ columns per row, only 5 of which are relevant to the user's question about quarterly revenue. The agent appends the full result set to context. After three such queries the window is nearly full and follow-up questions fail. What is the most effective fix?

- A Upgrade to a model with a larger context window so that full result sets can be accommodated across more queries.

A larger context window postpones the problem but does not solve it. Each full result set still wastes tokens on 35+ irrelevant columns, and the context will eventually fill regardless of size.

- B Limit the number of rows returned by each query to reduce the total data volume in context.

Row limits reduce data volume but may exclude relevant rows. The problem is column-level verbosity (35+ irrelevant columns per row), not row count. Trimming columns preserves all relevant rows whilst eliminating irrelevant fields.

- c** Trim tool results to only the relevant columns before appending them to the conversation context.

Tool result trimming removes irrelevant fields before they enter the context window. Keeping only the 5 relevant columns from each 40+ column result set dramatically reduces token consumption, allowing the agent to handle many more queries within the same context budget. (correct answer)

- d** Store all query results in an external database and have the agent retrieve specific values on demand instead of keeping results in context.

External storage adds infrastructure complexity without addressing the core issue. The agent still needs some result data in context to reason about it. Trimming irrelevant fields is simpler and directly reduces context consumption.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Tool result trimming)

Lesson 5.1: Context Window Management (Upstream optimisation)

Q42 · D3 3.2 SLASH-COMMANDS-SKILLS / COMMANDS-VS-SKILLS q-3-2-001 · Code Generation with Claude Code

SKIPPED

A team wants a `/review` command available to everyone who clones the repository. A developer also wants a personal `/brainstorm` skill that produces verbose codebase analysis output without cluttering the main conversation. Where should each be created and what configuration does the skill need?

- A** `/review` in `.claude/commands/` for team sharing; `/brainstorm` as a SKILL.md in `~/.claude/skills/` with context: fork frontmatter

`/review` needs to be project-scoped (`.claude/commands/` or equivalently `.claude/skills/`) so it is shared via version control. Both paths create identical `/commands`. `/brainstorm` is personal, so it goes in `~/.claude/skills/`. context: fork isolates the verbose analysis output from the main conversation. (correct answer)

- B** `/review` in `CLAUDE.md` as a documented procedure; `/brainstorm` in `.claude/skills/` with allowed-tools restrictions

`CLAUDE.md` is for always-loaded standards, not command definitions. The brainstorm skill needs context: fork for output isolation, not just allowed-tools.

- c Both in ~/.claude/commands/ with instructions for each developer to copy them locally

~/.claude/commands/ (or equivalently ~/.claude/skills/) is user-scoped and not shared via version control. Requiring manual copying defeats the purpose of project-scoped configuration.

- d Both in .claude/commands/ with the brainstorm skill using context: fork

The /brainstorm skill should be personal (not shared with the team), so it should not go in the project-scoped .claude/commands/ directory.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Where to place commands)

Lesson 3.2: Custom Slash Commands and Skills (Skills system)

Claude Code: Skills ↗

Q43 · D3 3.4 PLAN-MODE-EXECUTION / MODE-SELECTION q-3-4-001 · Code Generation with Claude Code **SKIPPED**

Your team faces three tasks: (1) restructure a monolith into microservices, (2) fix a null pointer exception in a single function with a clear stack trace, (3) migrate from one logging library to another across 30 files. Which mode should be used for each?

- A Plan mode for (1), direct execution for (2) and (3)

Task 3 affects 30 files and requires a consistent migration strategy. Without planning, you risk inconsistent application of the logging library change across files.

- B Plan mode for all three, since they all involve code changes

Task 2 is a well-understood, limited-scope bug fix. Plan mode would waste time for a change where the problem and solution are already clear.

- c Plan mode for (1) and (3), direct execution for (2)

Task 1 involves architectural decisions with multiple valid approaches. Task 3 affects 30 files with a pattern that needs design before execution. Task 2 is a single-function fix with a clear stack trace — direct execution is appropriate. (correct answer)

D Direct execution for all three with comprehensive upfront instructions

Task 1 (monolith restructuring) requires codebase exploration and architectural design. Comprehensive upfront instructions assume you already know the right structure without exploring.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan vs direct execution)

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Direct execution)

Q44 · D3 3.2 SLASH-COMMANDS-SKILLS / ALLOWED-TOOLS-SCOPING q-3-2-012 ·

Code Generation with Claude Code

SKIPPED

A junior developer is using Claude Code to refactor a payment processing module. The team lead wants to ensure Claude Code cannot accidentally delete production configuration files or modify the database migration directory during the refactoring session. What is the most appropriate approach?

- A Add a rule in CLAUDE.md stating 'Never modify files in the config/ or migrations/ directories'

CLAUDE.md instructions rely on the model's compliance and are not deterministic safeguards. For production-critical protection, tool-level permission restrictions provide reliable enforcement rather than natural language instructions.

- B Instruct the developer to use plan mode so Claude Code will ask for approval before each file change

Plan mode helps with evaluating strategies for complex tasks, but it does not provide file-level access control. Even in plan mode, the model can still access all tools and directories once execution begins.

- C Use `allowedTools` to limit the session to Read, Grep, Glob, and Write within the payment module

`allowedTools` provides tool-level permission control that can restrict which tools are available and their scope. By limiting write access to the payment module directory and keeping read access for context, the team lead ensures Claude Code cannot modify production config or migration files. (correct answer)

- D Set the whole repository to read-only at the filesystem level before the refactoring session starts.

Making the entire repository read-only would prevent Claude Code from making any changes, including the legitimate refactoring of the payment module. The requirement is selective protection, not blanket read-only access.

WHERE THIS COMES FROM

Claude Code: Settings ↗

Q45 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-010 ·

Content Moderation and Classification System

SKIPPED

A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

- A Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision. (correct answer)

- B Spawn a new combined text-and-image subagent for multi-modal reports so one specialist owns the whole decision rather than splitting it in two.

Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns.

- C Triage the whole report to whichever subagent handles the more severe category, so the moderation decision reflects the worst violation present.

Severity cannot be determined until both analyses are complete, so there is nothing to triage on. Routing to a single subagent also means one concern goes unanalysed.

- D Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.

Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

Q46 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-CONTEXT q-1-7-008 ·

Content Moderation and Classification System

SKIPPED

The moderation system handles appeals where users contest a moderation decision. Currently, the same agent that made the original decision re-evaluates the appeal. Appeal overturn rates are suspiciously low. What is the most effective architectural change?

- A Route every appeal to a human reviewer with full authority to overturn the automated decision, taking the original agent out of the appeal path entirely.

Human review for every appeal does not scale and removes the benefit of automated moderation. Some appeals can be resolved automatically by a separate, unbiased instance.

- B Add stronger instructions to the appeal handler's prompt requiring it to weigh the user's perspective fairly and set aside its earlier decision.

Prompt instructions cannot overcome the bias of an agent reviewing its own decision. The original reasoning context still influences the re-evaluation regardless of instructions.

- C Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.

A fresh agent instance without access to the original reasoning context evaluates the content independently. This avoids confirmation bias from the original decision. The appeal agent sees only the content and the user's argument, enabling a genuinely independent review. (correct answer)

- D Automatically overturn decisions where the user provides any appeal justification to improve user trust

Automatic overturn on any appeal completely undermines moderation. Users who genuinely violated policies would simply appeal every decision.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Q47 · D3 3.4 PLAN-MODE-EXECUTION / EXPLORE-SUBAGENT q-3-4-007

SKIPPED

A developer asks Claude Code to 'explain the order processing pipeline' before refactoring it. The pipeline spans 15 classes across 4 packages with deep inheritance and callback chains. Claude Code reads 3 files and produces a superficial summary that misses the callback chain entirely. What approach would produce a more thorough analysis?

- A Re-prompt Claude Code with more specific instructions listing all 15 classes to read

This assumes the developer already knows which 15 classes are involved. If the developer knew the full scope, they would not need Claude Code's help understanding the pipeline. The issue is that Claude Code's initial exploration was too shallow.

- B Switch to plan mode and ask Claude Code to create a plan for understanding the pipeline

Plan mode can safely explore a codebase before you commit to changes, but it runs in the main conversation and drives toward a change plan. Here the developer needs thorough, verbose discovery of current behaviour, which the Explore subagent isolates and summarises without exhausting the main context, so the Explore subagent is the better fit.

- C Run multiple Claude Code sessions, each analysing a different package, then manually combine the results

Splitting analysis across sessions loses the cross-package context needed to trace the callback chains and inheritance hierarchies. The pipeline's complexity comes from how the 4 packages interact, which requires a single comprehensive analysis.

- D Use an Explore subagent for verbose discovery across the pipeline's classes and callback chains

The Explore subagent is purpose-built for verbose, thorough codebase discovery. It systematically follows references, traces inheritance, and maps control flow across files — exactly what is needed for a complex event-driven pipeline. Regular Claude Code defaults to efficient, minimal reads; the Explore subagent prioritises comprehensive discovery. (correct answer)

WHERE THIS COMES FROM

## Lesson 3.4: Plan Mode vs Direct Execution (Explore subagent)

Q48 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ERROR-CATEGORIES q-2-2-006 · Multi-Agent Research System

SKIPPED

A `search\_papers` MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which `errorCategory` and `isRetryable` combination is correct for all three?

- A HTTP 503: transient, isRetryable true; restricted journal: business, false; malformed DOI: validation, false.

Only HTTP 503 is worth resending unchanged — a temporary upstream outage clears on its own. The restricted journal is a policy limitation (business) and the malformed DOI is a bad argument (validation); neither succeeds if the identical call goes out again, so both are isRetryable: false. The categories then split the recovery: the agent repairs the DOI itself, but a licence gap needs an alternative source or a human. (correct answer)

- B HTTP 503: transient, isRetryable true; restricted journal: transient, true; malformed DOI: transient, true.

The restricted journal error is not transient — the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; 'doi-1234' fails the same regex on every attempt, so resending it unchanged is wasted work.

- C All three are errorCategory: 'transient' with isRetryable: true, since each one stops the tool completing its task.

Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

- D HTTP 503: validation, isRetryable false; restricted journal: business, false; malformed DOI: business, false.

HTTP 503 is transient and worth resending — nothing is wrong with the request input, so marking it validation and non-retryable abandons a call that would have succeeded. A malformed DOI is validation, not business — business errors represent policy or rule violations, not input format problems.

WHERE THIS COMES FROM

## Lesson 2.2: Structured Error Responses (Four error categories)

Q49 · D5 5.4 CODEBASE-EXPLORATION / CRASH-RECOVERY q-5-4-011 · Code Generation with Claude Code

SKIPPED

A coordinator spawns four Claude Code subagents to explore a large codebase overnight: one maps the data layer, one traces API routes, one audits tests, one catalogues dependencies. Three hours in, the machine restarts. On rerun, the coordinator starts every exploration again from zero. What design change lets a restarted run continue from where the crash left off?

- A Run the four explorations sequentially in one session so all findings stay in a single context

Serialising the work gives up parallelism and pushes every verbose discovery into one context window, and a crash still loses the lot: an in-memory context is not durable state, however it is arranged.

- B Wrap each subagent in automatic retries so transient failures cannot end the run

Retries help an agent survive a failed tool call; they do nothing for a machine restart that kills the whole run. Recovery needs durable state that outlives the process, not more attempts within it.

- C Have each agent export structured state to a manifest the coordinator loads on resume

Structured state exports are the crash-recovery pattern: each agent persists its findings to a known location as it progresses, so after a failure the coordinator loads the manifest, sees what is complete, and seeds the remaining agents with prior findings instead of starting from zero. (correct answer)

- D Resume the coordinator's previous session so the conversation history is restored

Session resumption restores the coordinator's own conversation, not the subagents' accumulated findings: their contexts were separate and are gone. The coordinator would still have to re-run the explorations to regain that knowledge.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration (Crash recovery via structured state manifests)

Claude Docs: Agent SDK Sessions ↗

Q50 · D1 1.6 TASK-DECOMPOSITION / COORDINATOR q-1-6-007 · Multi-Agent Research System

SKIPPED

A multi-agent research system must process a customer's request that involves three sequential stages: data collection, analysis, and report generation. Each stage depends on the output of the previous one. Which orchestration pattern is most appropriate?

- A Hub-and-spoke with all three agents reporting independently to the coordinator

Independent reporting ignores the sequential dependencies. The analysis agent cannot produce meaningful results without data collection output. The pipeline must enforce the execution order.

- B Dynamic adaptive decomposition — let the coordinator decide the order at runtime based on query complexity

Dynamic adaptive decomposition is for open-ended investigation tasks where the next step depends on discoveries. Here, the three stages and their dependencies are known in advance, making a fixed pipeline the more predictable and efficient choice.

- C Pipeline orchestration — pass the output of each stage as input to the next in a defined sequence

Pipeline orchestration is the correct pattern for sequential dependencies. Each stage completes before the next begins, with the output of one stage serving as the input to the next. This matches the data collection → analysis → report generation dependency chain. (correct answer)

- D Parallel orchestration — run all three subagents simultaneously to minimise latency

Parallel execution requires independent tasks. Here, analysis depends on data collection results and report generation depends on analysis results. Running them simultaneously would mean the analysis agent has no data to analyse.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q51 · D2 2.1 TOOL-SCHEMA-DESIGN / DESCRIPTIONS q-2-1-010 ·

Enterprise Data Platform with Federated Queries

SKIPPED

The data platform's query\_snowflake tool has a description that reads: 'Queries Snowflake data warehouse. Accepts SQL.' The agent correctly uses the tool but frequently sends queries using PostgreSQL-specific syntax (e.g. string\_agg, which Snowflake spells LISTAGG) that Snowflake rejects. What is the most effective fix?

- A Have the MCP server automatically translate PostgreSQL syntax to Snowflake syntax before executing the query.

Auto-translation is fragile and cannot handle all dialect differences. It masks the problem rather than fixing it and introduces a complex transformation layer prone to edge cases.

- B Add a SQL syntax validation layer in front of the MCP tool that rejects non-Snowflake syntax before execution.

A validation layer adds infrastructure complexity and maintenance burden. The root cause is that the tool description does not specify which SQL dialect to use. Fixing the description is the proportionate first step.

- C Add a system prompt instruction listing all Snowflake-specific SQL functions the agent should use.

A system prompt instruction is brittle and consumes context tokens for every turn, even when the Snowflake tool is not being used. The tool description is the correct place for tool-specific dialect guidance.

- D State in the tool description that it expects the Snowflake SQL dialect, not PostgreSQL.

The description says 'Accepts SQL' without naming the dialect, so the agent defaults to PostgreSQL syntax. Naming the Snowflake dialect and giving examples in the description (LISTAGG rather than string\_agg, ILIKE is supported) resolves the misrouting at source. (correct answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

Q52 · D1 1.2 ORCHESTRATION-PATTERNS / ISOLATION q-1-2-009 · Content Moderation and Classification System

SKIPPED

The content moderation system uses a hub-and-spoke architecture with a coordinator that routes posts to specialist subagents: a text classifier, an image analyser, and a policy enforcer. The team notices that the image analyser sometimes directly calls the policy enforcer's action tools to remove posts, bypassing the coordinator. What is the architectural violation and how should it be fixed?

- A Add a message queue between the image analyser and the policy enforcer so the removal is decoupled and the coordinator can observe it after the fact.

An async queue between subagents still bypasses the coordinator, and observing a removal after it has

happened is not routing it. In hub-and-spoke, all inter-agent communication must flow through the coordinator regardless of synchronicity.

- B Merge the image analyser and policy enforcer into a single subagent to simplify the communication flow**

Merging subagents with distinct responsibilities (analysis vs enforcement) violates separation of concerns. The image analyser and policy enforcer have fundamentally different roles and tool requirements.

- C The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.**

In hub-and-spoke orchestration, subagents must be isolated and communicate only through the coordinator. The image analyser should return its classification result to the coordinator, which then decides whether to invoke the policy enforcer. Direct subagent-to-subagent communication breaks the architecture. (correct answer)

- D Give the image analyser its own copy of the policy enforcer's action tools so it can remove posts itself without calling another subagent.**

Duplicating action tools across subagents creates inconsistent enforcement and makes policy changes harder to propagate. The issue is the communication pattern, not tool availability.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

Lesson 1.2: Multi-Agent Orchestration (Hub-and-spoke architecture)

Q53 · D2 2.2 STRUCTURED-ERROR-RESPONSES / TRANSIENT-ERROR q-2-2-005 · Multi-Agent Research System

SKIPPED

A research agent calls an external API via an MCP server. After the 30th query in a batch of 50, the API starts returning HTTP 429 errors. The MCP server returns a generic 'Request failed' for every failure, so the agent abandons the batch after three consecutive failures. What MCP server change would most improve resilience?

- A Queue all 50 requests at the MCP server level and process them sequentially with built-in rate limiting.**

Server-side queuing removes the agent's ability to prioritise, cancel, or reorder queries based on intermediate results. It also creates a long-running blocking call that prevents the agent from doing other work.

- B Return `errorCategory`: 'business', `isRetryable`: false to tell the agent to stop making requests entirely.

Rate limits are transient (they resolve after a delay), not business errors (which represent policy or rule violations the agent cannot recover from). Marking them as non-retryable causes the agent to abandon 20 remaining queries that would succeed after a brief delay.

- C Return `errorCategory`: 'transient', `isRetryable`: true, with a `retryAfterMs` field telling the agent how long to wait.

Rate limiting is a transient error — it will resolve after a delay. Structured metadata with the specific retry delay lets the agent space out remaining queries intelligently rather than abandoning the batch. The agent can continue processing other tasks while waiting. (correct answer)

- D Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent entirely.

Hiding rate limits from the agent removes its ability to make informed decisions, such as prioritising remaining queries, switching to cached results, or reporting partial progress. The agent should receive structured metadata to decide its own recovery strategy.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Transient errors and retry-after)

Q54 · D5 5.5 HUMAN-REVIEW-CALIBRATION / AGGREGATE-METRICS-TRAP q-5-5-006 ·

Enterprise Data Platform with Federated Queries

SKIPPED

The data platform team runs a human review of the agent's federated query reports. Reviewers sample 50 reports and find an overall accuracy rate of 97%. Management approves the system for production. Three weeks later, users report that currency conversion calculations in cross-border revenue reports are wrong 40% of the time. How did the review process fail?

- A The reviewers were not domain experts in currency conversion and could not identify the errors.

Reviewer expertise is a valid concern but is not the structural failure. Even expert reviewers would miss the problem if the sample did not include enough cross-border reports. The sampling methodology is the root cause.

- B The agent's accuracy degraded over time due to model drift, and the initial 97% rate was genuinely correct at the time of review.

LLMs served via API do not experience gradual model drift within a three-week window. The error pattern was present during the review but hidden by aggregate metrics that did not stratify by report category.

- C The sample size of 50 reports was too small to detect a 40% error rate in currency conversions.

A 40% error rate in any category should be detectable in 50 reports — but only if the sample includes enough reports from that category. The problem is sampling strategy, not sample size.

- D The reviewers used aggregate accuracy metrics that masked category-specific failures such as the 40% currency error rate.

This is the aggregate metrics trap. When a low-frequency report type (cross-border revenue) has high error rates, the overall accuracy metric is dominated by high-frequency, easy report types. Stratified random sampling across report categories would have caught the currency conversion failures. (correct answer)

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Q55 · D5 5.6 INFORMATION-PROVENANCE / CONFLICT-HANDLING q-5-6-004 · Multi-Agent Research System

SKIPPED

A multi-agent research system finds conflicting data on a pharmaceutical compound's efficacy. A 2022 peer-reviewed clinical trial reports 78% efficacy, while a 2024 preprint reports 45% efficacy with a larger sample size. The synthesis agent must produce a recommendation for a medical advisory board. Which approach correctly handles the conflicting sources?

- A Exclude both findings and report that no reliable consensus exists, recommending further research before any conclusions

Excluding available evidence is unhelpful. The advisory board can make informed decisions with properly attributed conflicting data. Withholding findings because they conflict reduces the board's ability to assess the situation.

- B** Present both findings with full provenance: source identity, publication date, peer-review status, sample size, methodology, and any methodological differences that may explain the discrepancy. Let the advisory board weigh the evidence

For a medical advisory board, preserving full provenance is critical. The board needs to weigh peer-review status, sample size, methodology differences, and temporal context. Annotating both sources with complete metadata enables informed human judgment on conflicting scientific evidence. (correct answer)

- C** Use the 2024 preprint as it has a larger sample size and is more recent, noting it supersedes the 2022 study

Recency and sample size alone do not determine reliability. The preprint has not undergone peer review, and simply picking one source discards valuable context that the advisory board needs for informed decision-making.

- D** Average the two values (61.5%) and present it as the best estimate with a note about variance between studies

Averaging conflicting scientific results is methodologically invalid. The studies may have different populations, methodologies, or endpoints. The average has no scientific meaning and obscures the actual disagreement.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflicts intact)

**Q56** · D3 3.5 ITERATIVE-REFINEMENT / TEST-DRIVEN q-3-5-009 · Code Generation with Claude Code **SKIPPED**

A developer asks Claude Code to implement a rate limiter. Each iteration looks plausible, but edge cases keep surfacing in manual testing: burst traffic, clock skew, concurrent requests. Review comments fix one case and regress another. Which workflow change gives the iteration loop an objective target?

- A** Describe all the edge cases in more precise prose in a single detailed message

More precise prose still relies on the model interpreting descriptions consistently and on humans catching regressions by eye. Without executable checks, the loop has no objective pass/fail signal.

- B Switch to plan mode so the architecture is agreed before any implementation begins

Plan mode helps choose an approach for large or ambiguous changes, but the problem here is verification during iteration, not approach selection. A plan does not stop regressions from slipping through unreviewed edge cases.

- C Use the interview pattern so Claude asks clarifying questions before each iteration

The interview pattern surfaces unknown requirements before implementation in unfamiliar domains. These edge cases are already known; what is missing is a mechanical way to verify each iteration against them.

- D Write the test suite first, then iterate by sharing the failing test output

Test-driven iteration turns vague review comments into an objective, repeatable target: the failing tests define exactly what is wrong, and each iteration is judged by the same suite, which prevents fix-one-regress-another churn. (correct answer)

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Anthropic: Claude Code Best Practices ↗

Q57 · D5 5.3 ERROR-PROPAGATION / ACCESS-VS-EMPTY q-5-3-002 · Multi-Agent Research System

SKIPPED

A research subagent queries an academic journal database for articles on renewable energy policy. The database responds successfully but returns zero matching articles. Meanwhile, a second subagent querying a government statistics API receives a connection timeout. The coordinator currently handles both cases identically by retrying three times. What should change?

- A Increase the retry count to five for both cases to give transient failures more time to resolve

More retries do not fix the core problem. The journal database returned a valid empty result — it successfully searched and found nothing. Retrying it wastes time and API calls on an operation that already completed correctly.

- B Add exponential backoff to all retries so the subagents wait progressively longer between attempts

Backoff improves retry behaviour for transient failures but does not address the fundamental issue: the journal query does not need retries at all. It returned a valid response indicating no matches exist.

- C** Distinguish access failures from valid empty results: retry the API timeout as a transient failure, but accept the zero-match journal response as a valid finding and annotate the coverage gap

Access failures (timeouts, connection errors) indicate the tool could not reach the data source and warrant retry. Valid empty results mean the tool reached the source and found no matches — this IS the answer. Conflating these leads to unnecessary retries on valid results and missed coverage annotations. (correct answer)

- D** Remove retries entirely and immediately escalate both failures to a human operator for investigation

This is disproportionate. The API timeout is a transient failure that retries can resolve. The empty journal result is not a failure at all — it is a valid answer. Neither case requires human escalation.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Local recovery for transient)

Q58 · D2 2.4 MCP-SERVER-INTEGRATION / RESOURCES q-2-4-012 ·

Enterprise Data Platform with Federated Queries

SKIPPED

A data-analysis agent connects to an MCP server that exposes 40 database tables. Before almost every query, the agent makes several exploratory tool calls to discover which tables exist and what columns they hold, adding latency and token cost to each task. The server author wants to remove this discovery overhead. What is the most effective change?

- A** Enlarge the agent's context window so it can retain the results of the discovery calls across turns.

A larger context window does not stop the agent making the discovery calls; it still pays the latency and token cost on each task and merely stores the results.

- B** Add a `describe_schema` tool and instruct the agent in its system prompt to call it before every query.

This keeps an exploratory tool call on every task, which is the exact overhead the author wants to remove.

Resources present the catalogue without a per-task call.

- c Reduce the server to the five most frequently queried tables so there is less to discover.

This discards capability the agent legitimately needs and still leaves discovery calls for the remaining tables. It treats the symptom, not the discovery mechanism.

- d Expose the table catalogue and column schemas as MCP resources for upfront visibility.

MCP resources expose content catalogues such as database schemas, documentation hierarchies, and issue summaries, giving the agent visibility into available data upfront. The agent no longer needs to call `list_tables` and then `describe_table` for each table, which removes the discovery overhead entirely. (correct answer)

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (MCP resources)

Model Context Protocol: Resources ↗

Q59 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-CODED-CONTENT q-4-2-003 ·

Content Moderation and Classification System

SKIPPED

The moderation system classifies hate speech accurately for explicit slurs but misses coded language and dog-whistle terms that human moderators easily recognise. The prompt includes detailed written rules about coded language patterns. What intervention would most improve detection of coded hate speech?

- A Add 2-4 few-shot examples of coded hate speech, with reasoning naming the coded language and the targeted group.

Few-shot examples with reasoning tags teach the model the pattern-recognition process, not just individual terms. Showing the reasoning chain — surface meaning, coded meaning, targeted group, contextual signals — enables the model to generalise to new coded terms it has not seen before. (correct answer)

- B Add 20+ examples covering every known category of coded hate speech to maximise coverage

Laundry lists of examples dilute attention and teach pattern-matching rather than generalisation. 2-4 targeted examples with reasoning are more effective than 20 examples without reasoning because they teach the analytical process.

- C** Increase the model's context window so it can consider more of the user's post history for context

Context window size is not the bottleneck. The model fails to recognise coded language in individual posts, which is a prompt engineering issue, not a context limitation.

- D** Add a comprehensive dictionary of every known coded term and dog-whistle to the system prompt so the classifier can match posts against the full list.

A static dictionary becomes outdated immediately as coded language constantly evolves. It also bloats the prompt without teaching the model to recognise the underlying patterns of coded speech.

WHERE THIS COMES FROM

Lesson 4.2: Few-Shot Prompting (Few-shot for nuanced detection)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q60 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-017 · Multi-Agent Research System

SKIPPED

A web search agent has 9 tools: ``web_search``, ``url_fetch``, ``html_parse``, ``pdf_extract``, ``image_ocr``, ``translate``, ``summarise``, ``keyword_extract``, ``sentiment_analysis``. In testing it frequently calls ``summarise`` and ``sentiment_analysis`` when it should only fetch raw data. How should the architect fix this?

- A** Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.

The guide contrasts a focused 4-5-tool agent with an 18-tool one to show how selection reliability degrades as the set grows. The web search agent has 9 tools, causing it to use tools outside its intended role. Reducing to data-fetching tools enforces scoped access. The analysis tools (`summarise`, `sentiment_analysis`, `keyword_extract`) belong with the synthesis or analysis agents. (correct answer)

- B** Add system prompt instructions telling the web search agent to only use data fetching tools and ignore analysis tools

Prompt instructions are probabilistic and the agent is already misusing tools. The architectural fix is to remove the tools it should not have, providing deterministic enforcement through scoped tool access.

- C** Keep all 9 tools but implement PreToolUse hooks that block the web search agent from calling summarise and sentiment\_analysis

Using hooks to block tool access is an over-engineered workaround for what should be a configuration fix. The proper solution is to remove the tools from allowedTools, not to provide them and then block their use.

- D** Merge the web search and synthesis agents into a single agent since they share some tools

Merging agents violates the principle of specialised subagents. The solution is to separate tools by role, not to combine agents. Merging would create an even larger tool set on a single agent.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access (4-5 tools))

[Anthropic: Claude Agent SDK Overview](#) ↗

#### SUPPORT

### Found this useful?



[BUY ME A COFFEE](#)

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.